

Курс по JavaFX и SceneBuilder (интенсивная подготовка)

JavaFX — современная Java-платформа для создания графических настольных приложений, поддерживающая кроссплатформенную работу ¹. Она тесно интегрируется в IntelliJ IDEA: для работы с JavaFX необходимо включить плагин JavaFX ², задать JDK (Java 11+), а при сборке обеспечить подключение библиотек JavaFX (в новых версиях JavaFX вынесен из JDK и подключается отдельно) ³. **SceneBuilder** позволяет визуально проектировать интерфейс в FXML прямо в IDE ⁴. После установки плагина SceneBuilder можно открыть `.fxml` файл через функцию *Open In SceneBuilder* ⁵ или использовать его отдельное приложение ⁶ ⁵. Создав проект, IntelliJ сгенерирует пример приложения, и при успешном запуске можно приступить к разработке интерфейса ⁷ ⁸.

Общие понятия JavaFX

В JavaFX базовые понятия – **Stage** (окно) и **Scene** (сцена). Мы создаём корневой контейнер (например, `AnchorPane` или `GridPane`), задаём ему фон и элементы управления, затем «помещаем» эту панель в `Scene`. Цвет фона можно устанавливать через CSS-стиль или код. Например, `pane.setStyle("-fx-background-color: cornsilk");` задаст цвет `#FFF8DC` (Cornsilk) ⁹. Альтернативно: `scene.setFill(Color.CORNSILK)` или `pane.setBackground(new Background(new BackgroundFill(Color.CORNSILK, null, null)))`. JavaFX снабжён предопределёнными цветами (например, `Color.CADETBLUE` `#5F9EA0`, `Color.CORNSILK` `#FFF8DC` и др.) ¹⁰ ¹¹, а также утилитным методом `Color.rgb(r,g,b)` для произвольных RGB-значений ¹² или `Color.web("#RRGGBB")` для шестнадцатеричных кодов ¹³ ¹². Элементы управления (кнопки, метки, поля ввода и т.д.) можно размещать в макетах (VBox, HBox, GridPane, AnchorPane и др.). Для удобства разработки можно использовать FXML: вы проектируете интерфейс в SceneBuilder, а генерируемый `.fxml` файл связываете с контроллером Java (классом, который обрабатывает события).

Основные элементы управления

- **Слайдер (Slider)**. Позволяет выбрать числовое значение из диапазона. У слайдера есть ползунок («thumb») и полоса («track») ¹⁴. Например, `Slider slider = new Slider(0, 255, 255)` создаст горизонтальный слайдер с минимальным значением 0, максимальным 255 и стартовым 255. Можно настроить отображение меток и делений: `slider.setShowTickLabels(true)`, `slider.setMajorTickUnit(50)` и т.д. Значение слайдера получают через `slider.getValue()` или слушают изменения через `slider.valueProperty()`. При изменении положения ползунка можно реагировать слушателем:

```
slider.valueProperty().addListener((obs, oldVal, newVal) -> {
    // обновить метку newVal и цвет в реальном времени
});
```

Такой слушатель вызывается при любом изменении значения слайдера ¹⁵.

- **TextField и TextArea.** Поля ввода текста. У `TextField` есть метод `focusedProperty()`, к которому можно привязать `ChangeListener<Boolean>` – это позволит выполнять действия при потере или получении фокуса ¹⁶. Например:

```
textField.focusedProperty().addListener((obs, wasFocused, isNowFocused) -> {
    if (!isNowFocused) {
        // поле потеряло фокус – обновить изображения или валидацию
    }
});
```

При вводе текста или по нажатию Enter можно обрабатывать текст через события `onAction` или слушатели текста.

- **Кнопки (Button) и их события.** Для кнопок задаётся текст и можно настраивать цвет фона. В JavaFX это делается через CSS-стиль: например, `button.setStyle("-fx-background-color: lightblue");` задаст цвет `#ADD8E6` ¹⁷. Событие нажатия обрабатывается через `button.setOnAction(e -> { ... })`.

- **Checkbox и RadioButton.** `RadioButton` служит для выбора одного варианта из группы. Несколько радио-кнопок объединяются в `ToggleGroup` – тогда выбирается только одна из них ¹⁸. Чтобы поместить радио-кнопки в группу, делаем:

```
ToggleGroup group = new ToggleGroup();
radio1.setToggleGroup(group);
radio2.setToggleGroup(group);
radio1.setSelected(true); // выбрать по умолчанию
```

Проверить, какая кнопка выбрана, можно через `group.getSelectedToggle()` или `radio.isSelected()`. У `CheckBox` групп нет – можно отмечать несколько.

- **ImageView и загрузка изображений.** Чтобы показать картинку, используют `ImageView`. Сначала создают `Image image = new Image(new FileInputStream("путь"));`, затем `ImageView iv = new ImageView(image);`. У `ImageView` можно задать размеры: `iv.setFitWidth(100); iv.setFitHeight(100);`. Например:

```
Image image = new Image(new FileInputStream("obraz.png"));
ImageView iv = new ImageView(image);
iv.setFitWidth(200);
iv.setPreserveRatio(true);
```

Есть и способ загрузки из ресурсов: `new Image(getClass().getResourceAsStream("имя.jpg"))`. После создания, добавляем `iv` на панель. Пример загрузки изображения и вывода в сцену приведён на Tutorialspoint ¹⁹.

- **FileChooser (диалог выбора файла).** JavaFX предоставляет `FileChooser` для открытия/сохранения файлов. Пример использования для сохранения:

```
FileChooser fileChooser = new FileChooser();
fileChooser.setTitle("Сохранить файл");
File dest = fileChooser.showSaveDialog(stage);
if (dest != null) {
    Files.write(dest.toPath(), text.getBytes(StandardCharsets.UTF_8));
}
```

Метод `showSaveDialog(...)` показывает диалог сохранения и возвращает выбранный файл ²⁰. Можно использовать `java.nio.file.Files` для записи данных (строки или байтов) в этот файл ²⁰.

Теперь подробно разберём примерные приложения из экзаменационных заданий.

Задача 1. RGB-палитра (Wzornik kolorów RGB)

Описание интерфейса: Создайте окно с фоном **Cornsilk** (`#FFF8DC`), большим белым прямоугольником (`Rectangle`) и надписью «Dobierz kolor suwakami i zapisz przyciskiem:». Три горизонтальных слайдера (0–255, начальное значение 255) подписаны слева R, G, B, справа выводят текущее значение (например, через `Label`). Кнопка «Pobierz» имеет фон **Peru** (`#CD853F`). Внизу – небольшой белый прямоугольник с меткой текущих RGB (`"255, 255, 255"`).

Логика: Когда двигают любой слайдер, нужно: - Обновить число справа от него. Например:

```
sliderR.valueProperty().addListener((obs, oldV, newV) ->
    labelR.setText(String.format("%d", newV.intValue())));
```

- Установить цвет большого прямоугольника по текущим трем значениям. Код:

```
Color c = Color.rgb(sliderR.valueProperty().intValue(),
    sliderG.valueProperty().intValue(),
```

```
        sliderB.valueProperty().intValue());  
bigRect.setFill(c);
```

Здесь используется метод `Color.rgb(r,g,b)` для создания цвета из чисел 0–255 ¹².

При нажатии кнопки «**Pobierz**» в маленький прямоугольник нужно поместить выбранный цвет и вывести текст вида `"R, G, B"`:

```
buttonPobierz.setOnAction(e -> {  
    int r = (int)sliderR.getValue(), g = (int)sliderG.getValue(), b =  
    (int)sliderB.getValue();  
    Color c = Color.rgb(r,g,b);  
    smallRect.setFill(c);  
    labelSmall.setText(String.format("%d, %d, %d", r, g, b));  
});
```

После этого, дальнейшее движение слайдеров изменяет только большой прямоугольник и метки справа, но не маленький – для него цвет меняется лишь по новой команде кнопки.

Реализация (пример кода): UI можно задать либо в FXML (через SceneBuilder) или в коде. Ниже фрагмент обработчика для слайдеров:

```
sliderR.setMin(0); sliderR.setMax(255); sliderR.setValue(255);  
sliderG.setMin(0); sliderG.setMax(255); sliderG.setValue(255);  
sliderB.setMin(0); sliderB.setMax(255); sliderB.setValue(255);  
ChangeListener<Number> listener = (obs, oldV, newV) -> {  
    // обновить значения по каждому слайдеру  
    labelR.setText(String.valueOf((int)sliderR.getValue()));  
    labelG.setText(String.valueOf((int)sliderG.getValue()));  
    labelB.setText(String.valueOf((int)sliderB.getValue()));  
    // установить цвет большого прямоугольника  
    bigRect.setFill(Color.rgb((int)sliderR.getValue(),  
                             (int)sliderG.getValue(),  
                             (int)sliderB.getValue()));  
};  
sliderR.valueProperty().addListener(listener);  
sliderG.valueProperty().addListener(listener);  
sliderB.valueProperty().addListener(listener);
```

Фон окна задаётся стилем, например:

`rootPane.setStyle("-fx-background-color: cornsilk");` ⁹. Поля меток можно стилизовать белым фоном. Код форматирования и названия переменных должны быть «чистыми» и понятными.

Задача 2. Шифрование текста (Szyfrowanie)

Описание интерфейса: Окно с фоном **CadetBlue** (`#5F9EA0` ²¹), заголовок **Szyfrowanie**. Wykonane przez <номер>. На нём три надписи цветом **AntiqueWhite** (`#FAEBD7` ²²): «Podaj wartość klucza», «Podaj tekst», «Tekst zaszyfrowany». Поля ввода: однострочный **TextField** для ключа, многострочный **TextArea** для ввода текста (с переносом строк), и **TextArea** (или **Label**) для зашифрованного текста. Последнее поле стилизовать так: фон линии – **AliceBlue** (`#F0F8FF` ²³ с обводкой **AntiqueWhite** и закруглёнными углами). Кнопки («Zaszyfruj», «Zapisz szyfr w pliku») с фоном **LightBlue** (`#ADD8E6` ¹⁷).

Логика: При нажатии «Zaszyfruj» берём введённый ключ и текст, шифруем и выводим результат. Если в поле ключа введено нечисло, принимаем ключ = 0. Пример простого шифрования – **шифр Цезаря** (Caesar cipher): каждую букву сдвигаем на k позиций в алфавите. Например, сдвиг на 3: A→D, B→E, ..., Z→C ²⁴. На практике:

```
int key;
try {
    key = Integer.parseInt(keyField.getText());
} catch (NumberFormatException ex) {
    key = 0;
}
String text = inputArea.getText();
StringBuilder sb = new StringBuilder();
for (char ch : text.toCharArray()) {
    if (Character.isUpperCase(ch)) {
        char c = (char)('A' + (ch - 'A' + key) % 26);
        sb.append(c);
    } else if (Character.isLowerCase(ch)) {
        char c = (char)('a' + (ch - 'a' + key) % 26);
        sb.append(c);
    } else {
        sb.append(ch); // не буква остается без изменений
    }
}
outputArea.setText(sb.toString());
```

(Для упрощения можно не учитывать дополнительные символы или национальные буквы).

Основная идея шифра Цезаря – сдвиг алфавита на *offset* ²⁴.

При нажатии «Zapisz szyfr w pliku» открывается диалог сохранения (FileChooser). Код может быть таким:

```
FileChooser chooser = new FileChooser();
chooser.setTitle("Zapisz szyfr");
File dest = chooser.showSaveDialog(primaryStage);
```

```

if (dest != null) {
    try {
        Files.write(dest.toPath(),
            outputArea.getText().getBytes(StandardCharsets.UTF_8));
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
}

```

Метод `showSaveDialog` возвращает `File`, в который записываем зашифрованный текст ²⁰. Альтернативно можно использовать `Files.copy(...)` или `FileWriter`. Главное – сохранить `outputArea.getText()` в выбранный файл ²⁰.

Стили: Цвета надписей и фонов задаются так: - Фон окна: `CadetBlue` (`rootPane.setStyle("-fx-background-color: cadetblue")`) ²¹. - Текст надписей: `label.setTextFill(Color.ANTIQUEWHITE)` ²². - Цвет текста зашифрованного поля: `Color.ALICEBLUE` ²³. - Кнопки: `button.setStyle("-fx-background-color: lightblue; -fx-font-weight: bold")` ¹⁷. - Границы полей можно сделать через CSS или классы (`setStyle("-fx-border-color: antiquewhite; -fx-background-radius: 10")`).

Задача 3. Приложение «Моя музыка» (MojeDźwięki)

Описание интерфейса: Окно «MojeDźwięki, Wykonał: <номер>» с фоном `SeaGreen` (`#2E8B57`) ¹¹). В центре – изображение виниловой пластинки (`ImageView`) из файла `obraz.png`. Рядом два кнопки-навигации (влево/вправо) с графикой (`obraz2.png`, `obraz3.png`) фиксированной высоты 70. Под ними расположены пять меток (`Label`) для: «имя исполнителя», «название альбома», «кол-во треков», «год выпуска», «кол-во скачиваний». Шрифт: цвет белый и ярко-зелёный (`#61D918`), размеры 50, 30, 20; название альбома курсивом. Кнопка «Pobierz» фоновым цветом `#61D918`, белым жирным шрифтом 20px.

Логика: При запуске читаем файл `Dane.txt` (он должен быть рядом). Предположим, каждая строка содержит данные об альбоме (например, разделённые `;` или другим разделителем). Пример формата:

```

Gorillaz;Gorillaz;18;2010;52
Melanie Martinez;Cry Baby;20;2015;37
...

```

(где последняя цифра – скачивания). Загружаем эти записи в список объектов (например, `Album{author,title,tracks,year,downloads}`). Далее показываем данные первого альбома: заполняем метки соответствующими полями и изображение (`vinylImage.setImage(vinylImg)`).

При нажатии «влево» (`prevButton`): уменьшаем индекс текущего альбома (или переходим к последнему, если был первый). При «вправо»: увеличиваем индекс (или к первому из последнего). Затем обновляем отображение:

```

Album alb = albums.get(currentIndex);
labelAuthor.setText(alb.author);
labelTitle.setText(alb.title);
labelTracks.setText(String.valueOf(alb.tracks));
labelYear.setText(String.valueOf(alb.year));
labelDownloads.setText(String.valueOf(alb.downloads));

```

При нажатии «Pobierz»: просто увеличиваем поле `alb.downloads++` и обновляем `labelDownloads`. Эти изменения хранятся в оперативной памяти (в массиве `albums`)²⁵; при закрытии приложения они не записываются обратно (по условию «не требуется»).

Цвета и стили: Фон окна `SeaGreen`¹¹. Текст меток белый (`Color.WHITE`²⁶) и зелёный (`Color.web("#61D918")` через CSS или код). Курсив для названия альбома можно задать `labelTitle.setFont(Font.font("System", FontPosture.ITALIC, 30))`. Кнопки – просто `ImageView` с `Button.setGraphic()`.

Задача 4. Паспортные данные (Wprowadzenie danych do paszportu)

Описание интерфейса: Окно «Wprowadzenie danych do paszportu. Wykonał: <номер>» с фоном `CadetBlue` (`#5F9EA0`)²¹. Есть три текстовых поля (`TextField`): с надписями «Numer», «Imię», «Nazwisko». Группа радио-кнопок «Kolor oczu» с вариантами «niebieskie», «zielone», «piwne» (обычно реализуем через `ToggleGroup`). По умолчанию отмечено «niebieskie». Кнопка «OK». Два изображения (`ImageView`): они из файлов `<номер>-zdjecie.jpg` и `<номер>-odcisk.jpg` (например, `000-zdjecie.jpg`). Высота обоих 180.

Логика: При выходе из поля «Numer» (фокус ушёл, можно слушать через `focusedProperty`)¹⁶ нужно обновить изображения: строим имя файла из номера. Например:

```

String num = numerField.getText();
try {
    Image photo = new Image(new FileInputStream(num + "-zdjecie.jpg"));
    imageViewPhoto.setImage(photo);
} catch (FileNotFoundException ex) {
    imageViewPhoto.setImage(null);
}
try {
    Image finger = new Image(new FileInputStream(num + "-odcisk.jpg"));
    imageViewFinger.setImage(finger);
} catch (FileNotFoundException ex) {
    imageViewFinger.setImage(null);
}

```

Если файла нет, `setImage(null)` оставит поле пустым (или можно не менять).

По нажатию «ОК»: проверяем, введены ли имя и фамилия. Если одно из полей пусто, показываем предупреждение (например,

`Alert alert = new Alert(AlertType.WARNING, "Wprowadź dane"); alert.show();`). Иначе выводим сообщение вида: `"ИМЯ ФАМИЛИЯ kolor oczu КОЛОР"`. Например:

```
String imie = imieField.getText(), nazw = nazwField.getText();
if(imie.isEmpty() || nazw.isEmpty()){
    // Показать ошибку
} else {
    String eye = (radioBlue.isSelected() ? "niebieskie" :
        radioGreen.isSelected() ? "zielone" : "piwne");
    String msg = String.format("%s %s kolor oczu %s", imie, nazw, eye);
    Alert alert = new Alert(AlertType.INFORMATION, msg);
    alert.show();
}
```

(В PolishLocale можно использовать `showAndWait()`).

Стили: Фон CadetBlue ²¹. Поля и кнопка Azure (#F0FFFF) ²⁷ задаются стилем `-fx-background-color: azure;`. Группа радио может быть на `VBox` или `GroupBox`. Имена и названия файлов (`000-zdjecie.jpg`) указаны в условии; для тестов нужно добавить все файлы из архива. При неправильном номере изображение не выводится.

Задача 5. Отправка посылки (Nadaj Przesyłkę)

Описание интерфейса: Диалоговое окно «Nadaj Przesyłkę [номер]». Группа радиокнопок: «Pocztówka», «List», «Paczka» (ToggleGroup), по умолчанию – «Pocztówka». Три поля `TextField` для «Ulica z numerem», «Kod pocztowy», «Miasto» (можно объединить в `VBox`). Кнопка «Sprawdź cenę». Под ней `ImageView` (показывает картинку: по умолчанию `pocztowka.png`). Рядом метка «Cena: » с крупным жирным шрифтом. Далее кнопка «Zatwierdź».

Логика: При выборе радио-кнопки и нажатии «Sprawdź cenę» нужно поменять картинку и цену: - Если **Pocztówka**: `imageView.setImage(new Image(..."/pocztowka.png"));` `labelCena.setText("Cena: 1 zł");` - **List**: `list.png`, «Cena: 1,5 zł». - **Paczka**: `paczka.png`, «Cena: 10 zł».

Примерно так:

```
checkButton.setOnAction(e -> {
    Toggle t = group.getSelectedToggle();
    if (radioPoczt.isSelected()) {
        imageView.setImage(new
Image(getClass().getResourceAsStream("pocztowka.png")));
        priceLabel.setText("Cena: 1 zł");
    }
```



```

    } else if (radiolist.isSelected()) {
        imageView.setImage(new
Image(getClass().getResourceAsStream("list.png")));
        priceLabel.setText("Cena: 1,5 zł");
    } else if (radioPaczka.isSelected()) {
        imageView.setImage(new
Image(getClass().getResourceAsStream("paczka.png")));
        priceLabel.setText("Cena: 10 zł");
    }
});

```

При нажатии «Zatwierdź» валидируем поле «Kod pocztowy»: в условии сказано – только 5 цифр без «-». Реализуем так:

```

String code = codeField.getText();
if (code.length() != 5) {
    alert("Nieprawidłowa liczba cyfr w kodzie pocztowym");
} else if (!code.matches("\\d{5}")) {
    alert("Kod pocztowy powinien się składać z samych cyfr");
} else {
    alert("Dane przesyłki zostały wprowadzone");
}

```

(где `alert(msg)` выводит, например, `Alert` или `showMessageDialog`).

Стили: Картинки лежат в ресурсах. Метка «Cena: » можно сделать `Label priceLabel = new Label("Cena: "); priceLabel.setStyle("-fx-font-size: 20px; -fx-font-weight: bold");`. Цвета не указаны, поэтому используем стандартные (белый текст на кнопках по желанию). Главное – логика нажатий и проверок.

Задача 6. Добавление сотрудника (Dodaj pracownika)

Описание интерфейса: Окно «Dodaj pracownika [номер]» с фоном **LightSteelBlue** (`#B0C4DE` ²⁸). Раздел «Dane Pracownika»: два поля `TextField` с метками «Imię» и «Nazwisko», выпадающий список (`ChoiceBox` или `ComboBox`) «Stanowisko» со значениями «Kierownik, Starszy programista, Młodszy programista, Tester». Раздел «Generowanie hasła»: поле `TextField` «Ile znaków?», три чекбокса «Małe i wielkie litery», «Cyfry», «Znaki specjalne» (первый отмечен по умолчанию), кнопка «Generuj hasło». Внизу кнопка «Zatwierdź» (длиннее кнопки генерации). Кнопки с фоном **SteelBlue** (`#4682B4` ²⁹) и белым текстом (`Color.WHITE` ²⁶).

Логика: При «Generuj hasło» генерируем пароль заданной длины. Базовый набор – маленькие латинские буквы (`a-z`). Если отмечен чекбокс «Małe i wielkie litery», добавляем в пароль хотя бы одну большую букву (`A-Z`). Если «Cyfry» – хотя бы одну цифру (`0-9`), «Znaki specjalne» – хотя бы один символ из `!@#$%^&*()_+~=`. Упрощение: можно всегда фиксированно добавить одну цифру и один

спецсимвол, как указано в условии (например, первый символ пароля – цифра, второй – спецсимвол, остальные – буквы). Пример кода:

```
int len = Integer.parseInt(lengthField.getText());
StringBuilder pw = new StringBuilder();
Random rnd = new Random();
// Всегда добавим одну большую букву, если нужно:
if (bigLettersChk.isSelected()) {
    pw.append((char)('A' + rnd.nextInt(26)));
}
if (digitsChk.isSelected()) {
    pw.append(rnd.nextInt(10)); // цифра 0-9
}
if (specialChk.isSelected()) {
    String specials = "!@#$%^&*()_+!=";
    pw.append(specials.charAt(rnd.nextInt(specials.length())));
}
// Заполнить остальными маленькими буквами:
while (pw.length() < len) {
    pw.append((char)('a' + rnd.nextInt(26)));
}
// Перемешать символы, если нужно, или оставить:
String password = pw.toString();
```

Выводим пароль в `Alert` или `TextField`. Например:

```
Alert info = new Alert(AlertType.INFORMATION, "Wygenerowane hasło: " +
password);
info.showAndWait();
```

(В условии сказано вывести в сообщении по образцу на рисунке). Пакет символов: `a-z` (ASCII 97-122), `A-Z` (65-90), `0-9` (48-57), спецсимволы как строка. В примерах генерации случайной строки демонстрируется подход через `Random` ³⁰.

При «Zatwierdź» выводим заполненные данные: `Imię`, `Nazwisko`, `Stanowisko` и сгенерированный пароль. Например:

```
String infoMsg = String.format("Pracownik: %s %s, Stanowisko: %s, Hasło: %s",
    imieField.getText(), nazwField.getText(), stanowiskoChoice.getValue(),
password);
new Alert(AlertType.INFORMATION, infoMsg).show();
```

Валидацию полей в упрощённом задании можно опустить (по условию).

Стили: Фон LightSteelBlue ²⁸ . Кнопки:

```
okButton.setStyle("-fx-background-color: steelblue; -fx-text-fill: white; -fx-font-size: 14px;");
genButton.setStyle("-fx-background-color: steelblue; -fx-text-fill: white;");
```

(Цвет SteelBlue указан в [35†L672-L674]). Это задаст нужные цвета. Поля ввода оставить белыми или Azure по умолчанию.

Заключение

В этом курсе мы рассмотрели создание десктоп-приложений на JavaFX с помощью SceneBuilder и программного кода. В примерах показано, как работать со слайдерами и динамически менять цвет ¹² ⁹ , обрабатывать текстовое шифрование и сохранять файл ²⁰ ²⁴ , делать навигацию по данным из файла, обновлять `ImageView` при вводе данных ¹⁹ ¹⁶ , работать с группами радиокнопок ¹⁸ ³¹ и чекбоксов, а также генерировать случайные пароли. Каждый пример снабжён подробным описанием интерфейса и действий. Код приложений должен быть оформлен по правилам «чистого кода» (понятные имена, структурированное форматирование). Для дальнейшего углубления рекомендуется изучить официальные руководства и документацию JavaFX ³² ⁴ .

Источники: В курсе использованы официальные материалы JavaFX (Oracle и JetBrains), а также авторитетные учебные ресурсы ¹ ⁴ ¹⁴ ²⁰ ¹² . Все цвета и константы получены из справочника `javafx.scene.paint.Color` ¹⁰ ²⁸ ²⁹ . Код шифрования иллюстрирует подход шифра Цезаря ²⁴ , а сохранение файла – использование `FileChooser` ²⁰ .

¹ ² ³ ⁷ ⁸ ³² Create a new JavaFX project | IntelliJ IDEA Documentation

<https://www.jetbrains.com/help/idea/javafx.html>

⁴ ⁵ ⁶ Configure JavaFX Scene Builder | IntelliJ IDEA Documentation

<https://www.jetbrains.com/help/idea/opening-fxml-files-in-javafx-scene-builder.html>

⁹ java - JavaFX Scene/background Color: How to Change? - Stack Overflow

<https://stackoverflow.com/questions/71328159/javafx-scene-background-color-how-to-change>

¹⁰ ¹¹ ¹² ¹³ ¹⁷ ²¹ ²² ²³ ²⁶ ²⁷ ²⁸ ²⁹ Color (JavaFX 8)

<https://docs.oracle.com/javase/8/javafx/api/javafx/scene/paint/Color.html>

¹⁴ ¹⁵ Using JavaFX UI Controls

https://docs.oracle.com/javafx/2/ui_controls/jfxpub-ui_controls.pdf

¹⁶ java - How perform task on JavaFX TextField at onfocus and outfocus? - Stack Overflow

<https://stackoverflow.com/questions/16549296/how-perform-task-on-javafx-textfield-at-onfocus-and-outfocus>

¹⁸ ²⁵ ³¹ Using JavaFX UI Controls: Radio Button | JavaFX 2 Tutorials and Documentation

https://docs.oracle.com/javafx/2/ui_controls/radio-button.htm

¹⁹ JavaFX - Images

https://www.tutorialspoint.com/javafx/javafx_images.htm

20 How to save a file using FileChooser in JavaFX - Stack Overflow

<https://stackoverflow.com/questions/34696417/how-to-save-a-file-using-filechooser-in-javafx>

24 The Caesar Cipher in Java | Baeldung

<https://www.baeldung.com/java-caesar-cipher>

30 Java - Generate Random String | Baeldung

<https://www.baeldung.com/java-random-string>